

UNIVERSIDAD NACIONAL DE INGENIERÍA

FACULTAD DE INGENIERÍA INDUSTRIAL Y DE SISTEMAS



INFORME TÉCNICO

SCOTIABANK

Asignatura: Sistema de Inteligencia de Negocios

Sección: SI807-U

Grupo: 6

Integrantes:

Álvarez Casas Julio Cesar	20224036C
Campos Herrera Dennis Leopoldo	20220091J
Espinal Aguilar, Albert Moisés	20220430I

2025-II

ÍNDICE

1.	Arquitectura Cloud definida	4
2.	Los servicios utilizados.	4
2.1.	Servicios y APIs habilitadas	5
2.2.	Procesamiento y preprocesamiento de datos	6
2.3.	Evidencia de uso de recursos y costos del entorno.....	7
3.	Diseño del Data Lake	8
3.1.	Diseño y enfoque adoptado	8
3.2.	Organización lógica de la capa Raw (GCS)	9
3.3.	Implementación de las capas Bronce, Plata y Oro (BigQuery).....	10
4.	El modelo dimensional implementado	10
4.1.	Dimensión Fecha (dim_fecha).....	10
4.2.	Dimensión Moneda (dim_moneda)	11
4.3.	Dimensión Banco (dim_banco)	11
4.4.	Dimensión Indicador (dim_indicador)	12
4.5.	Tabla de hechos de eventos de riesgo (hecho_riesgo).....	13
5.	Procesos ETL.....	15
5.1.	ETL inicial y carga a la capa Bronce	15
5.2.	Transformación y depuración en la capa Plata	15
5.3.	Construcción de la capa Oro.....	15
6.	Los scripts utilizados y consultas SQL.....	16

6.1.	Servicios GCP	16
6.2.	Medallion.....	18
7.	Visualizaciones Finales Implementadas	22
8.	El uso del repositorio GitHub.....	26
	BIBLIOGRAFÍA	27

1. Arquitectura Cloud definida

El entorno de trabajo fue desplegado íntegramente en Google Cloud Platform (GCP), utilizando servicios administrados para garantizar escalabilidad, disponibilidad y reducción de carga operativa.

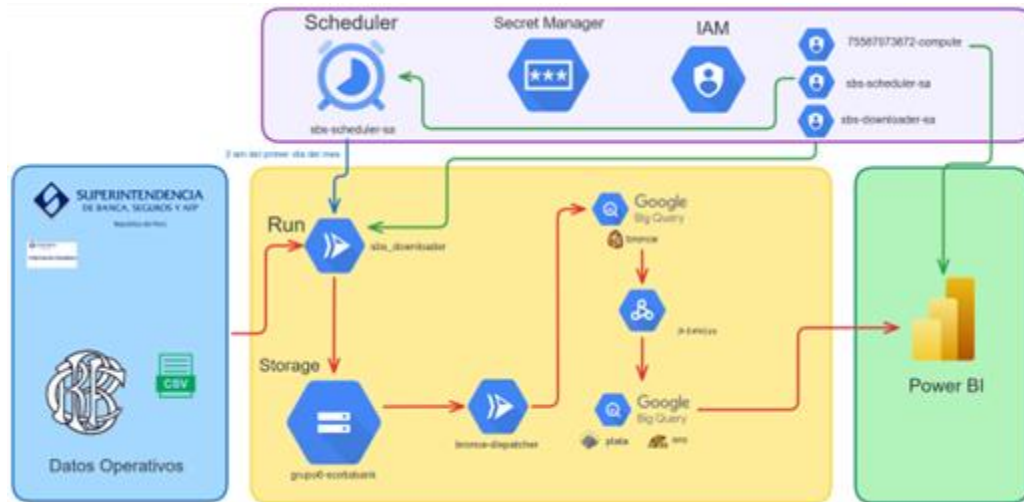


Figura 1. Arquitectura desarrollada

La arquitectura fue diseñada en base al procesamiento siguiendo la metodología Medallion por lo que primero buscamos un acoplamiento con la fuente de datos externas (SBS y BCRP) a través de un servicio barato y fácil de implementar y resultamos usando Cloud Run como opción asociada a un Scheduler.

De igual manera se usaron las capacidades de procesamiento de GCP en cada uno de los layers, y a pesar de que al inicio se consideró usar Looker por ser la herramienta nativa de GCP para el establecimiento de tableros BI esta opción se descartó al no ser replicable para un usuario nuevo que no tenga acceso a la herramienta ya implementada, por eso se optó por el uso de Power BI.

La integración de los componentes se estableció a través de medios seguros como las cuentas de servicio para cada función y una división operativa en 3 roles con permisos únicos asignados.

2. Los servicios utilizados.

Cloud Storage se empleó como repositorio central del Data Lake, organizando los datos en buckets estructurados por capas (Bronce, Plata y Oro). Cloud Functions fue utilizada para ejecutar los procesos de ingesta automática de datos, activados de forma programada mediante Cloud Scheduler. Para el procesamiento distribuido de grandes volúmenes de datos se utilizó Dataproc, ejecutando jobs PySpark bajo demanda. Finalmente, BigQuery se configuró como el motor analítico central, permitiendo consultas SQL eficientes y su integración directa con Power BI.

La estructura de servicios utilizada se fue desarrollando en los entregables anteriores al actual informe técnico, donde se desarrolló la justificación de la plataforma y el proveedor para el caso y el despliegue de recursos y servicios necesarios.

2.1. Servicios y APIs habilitadas

Para el funcionamiento integral de la solución, fue necesario la habilitación de diversas APIs de Google Cloud, las cuales permitieron la comunicación entre servicios, la ejecución de procesos serverless, el procesamiento distribuido, el almacenamiento analítico y el monitoreo del sistema. Estas APIs soportan la automatización de los flujos de ingesta y procesamiento, así como la observabilidad y control operativo de la arquitectura. La relación detallada de APIs habilitadas y sus métricas operativas se presenta en la tabla correspondiente.

Tabla 1. APIs habilitadas para el proyecto

API	Descripción breve del servicio	Solicitudes	Errores (%)	Latencia mediana (ms)	Latencia p95 (ms)
Cloud Logging API	Recolección, almacenamiento y consulta centralizada de logs del sistema	25,046	0	59	122
Cloud Dataproc Control API	Gestión y control de clústeres y jobs de procesamiento distribuido	7,488	3	30	64
Cloud Monitoring API	Monitoreo de métricas, recursos y estado operativo del entorno	1,440	0	24	32
Compute Engine API	Administración de recursos de cómputo subyacentes a Dataproc	422	0	98	220
BigQuery API	Ejecución de consultas analíticas y gestión de datasets y tablas	70	0	160	1,805
Cloud Run Admin API	Administración de servicios serverless y despliegue de funciones	56	0	50	214
BigQuery Storage API	Acceso de alta velocidad a datos de BigQuery para procesos externos	52	16	376	878
Eventarc API	Enrutamiento de eventos para arquitecturas orientadas a eventos	22	0	221	476

Cloud Pub/Sub API	Mensajería asíncrona para integración entre servicios	21	0	196	255
Artifact Registry API	Gestión y versionado de artefactos y dependencias del sistema	12	0	54	117
Cloud Functions API	Ejecución de funciones serverless para ingesta y automatización	5	0	196	491
Cloud Storage API	Acceso programático y gestión de objetos del Data Lake	4	0	196	255

2.2. Procesamiento y preprocesamiento de datos

El procesamiento y preprocesamiento de los datos se implementó mediante una Cloud Function denominada *bronze-dispatcher*, la cual se ejecuta cada vez que se detecta la carga de un nuevo archivo en un bucket de Google Cloud Storage. Esta función opera bajo un esquema *event-driven*, reaccionando a eventos de tipo *object finalize* generados por el almacenamiento en la nube.

Al recibir el evento, la función *bronze-dispatcher* valida el origen del archivo y evalúa la ruta de almacenamiento para identificar el tipo de reporte financiero ingresado. En función de esta clasificación, el dispatcher invoca dinámicamente el pipeline ETL correspondiente, el cual ejecuta tareas específicas de formateo y normalización inicial de los datos. En esta etapa no se aplican reglas de negocio ni transformaciones analíticas complejas, ya que el objetivo principal es convertir los archivos de entrada en estructuras tabulares estandarizadas, compatibles con su posterior procesamiento distribuido.

Los datos resultantes de este proceso inicial se almacenan en la capa *Bronce*, preservando el contenido original ya normalizado y garantizando la disponibilidad de una versión controlada y auditable de la información.

Posteriormente, los datos almacenados en la capa *Bronce* son procesados en la capa *Plata* mediante jobs PySpark ejecutados sobre Dataproc, donde se aplican transformaciones orientadas a la depuración y estandarización avanzada. Estas transformaciones incluyen la eliminación de registros duplicados, la validación de integridad de los datos, la estandarización de tipos de datos y la armonización de estructuras provenientes de distintos reportes regulatorios.

Tabla 2. Configuración de recursos del job PySpark en Dataproc

Componente	Configuración
Plataforma de ejecución	Google Cloud Dataproc

Tipo de job	PySpark Batch
Versión runtime	Dataprocc 2.1
Número de ejecutores	2
Núcleos por ejecutor	4
Memoria por ejecutor	4 GB
Núcleos del driver	4
Memoria del driver	4 GB
Conector BigQuery	spark-bigquery-latest_2.12.jar

La ejecución de estos jobs es orquestada automáticamente por la Cloud Function bronce-dispatcher, la cual, se lanza después de ejecutar el pipeline ETL. El uso de PySpark y Dataprocc permite ejecutar estas operaciones de forma distribuida y paralela, garantizando tiempos de procesamiento adecuados para volúmenes de datos característicos del sector financiero.

Finalmente, la información depurada en la capa Plata es consolidada en la capa Oro, donde se construye un modelo analítico de tipo estrella, compuesto por tablas de dimensiones y una tabla de hechos. Esta capa representa la versión final y confiable de los datos, optimizada para el análisis multidimensional. Los datos de todas las capas se encuentran en BigQuery, lo que permite su disponibilidad inmediata para consultas e integración con otras herramientas.

2.3. Evidencia de uso de recursos y costos del entorno

Como evidencia del uso real de los servicios desplegados, se presenta el resumen de costos generado por Google Cloud Billing. Los registros muestran consumo asociado a BigQuery, Cloud Storage, Compute Engine y Cloud Run Functions, entre otros servicios. Este resultado confirma la viabilidad de la solución bajo un esquema de bajo costo para entornos académicos y de prototipado.

Servicio	Costo por uso ?	Programas de ahorros ?	Otros ahorros ?	↓ Subtotal	% de cambio ?
▼ BigQuery	S/. 5.65	—	-S/. 5.65	S/. 0.00	—
◆ Cloud Run Functions	S/. 0.05	—	-S/. 0.05	S/. 0.00	—
■ Cloud Build	S/. 0.00	—	S/. 0.00	S/. 0.00	—
● Networking	S/. 3.19	—	-S/. 3.19	S/. 0.00	—
✦ BigQuery Reservation API	S/. 12.88	—	-S/. 12.88	S/. 0.00	—
■ Cloud Logging	S/. 0.00	—	S/. 0.00	S/. 0.00	—
▲ Cloud Storage	S/. 20.55	—	-S/. 20.55	S/. 0.00	—
✱ Compute Engine	S/. 180.60	—	-S/. 180.60	S/. 0.00	—

Figura 2. Desglose de costos por servicios

Hasta la fecha el consumo total del monto de la versión de prueba se estima en un aproximado de S/. 220 por un periodo de 2 meses utilizados. De esta forma se puede visualizar un resumen de los costos implicados en el uso de los servicios:

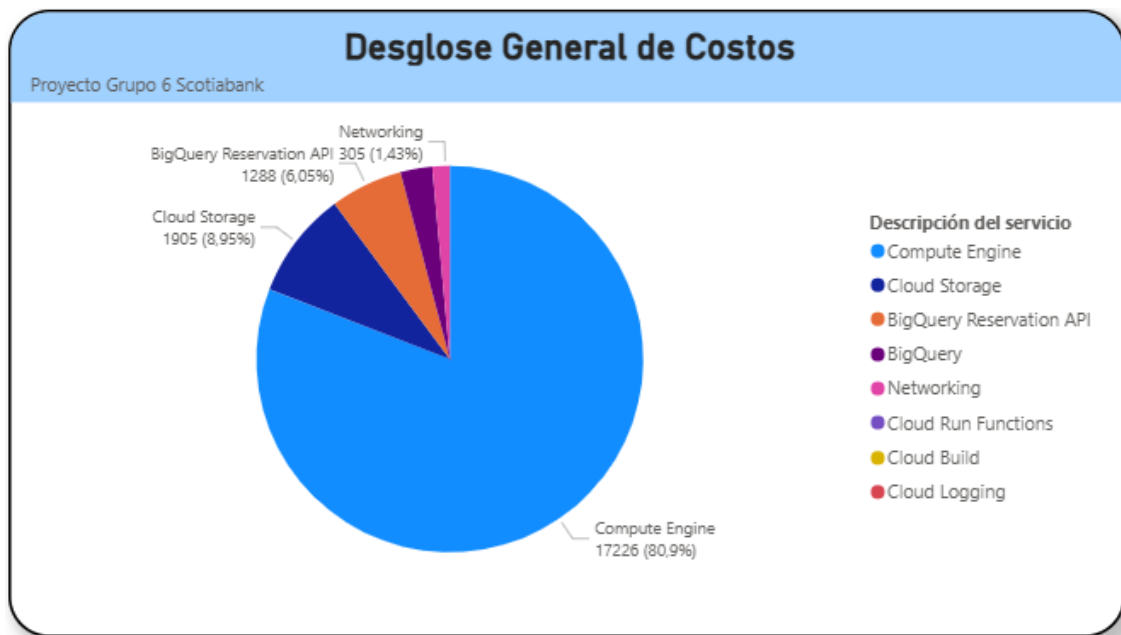


Figura 3. Gráfico de porcentaje de costos del proyecto.

La Figura 3 presenta el desglose general de costos del proyecto, evidenciando la distribución relativa del consumo de servicios en Google Cloud Platform. Se observa que Compute Engine concentra la mayor proporción del costo total con aproximadamente 80,9%, lo cual está directamente asociado al uso intensivo de recursos de cómputo para la ejecución de jobs PySpark en Dataproc, responsables del procesamiento y transformación de grandes volúmenes de datos.

En segundo lugar, Cloud Storage representa aproximadamente el 9% del consumo, reflejando el almacenamiento persistente de datos en las distintas capas del Data Lake (Bronce, Plata y Oro). Asimismo, el BigQuery Reservation API aporta cerca del 6%, vinculado al uso de capacidades analíticas y consultas sobre la capa Oro.

Los costos asociados a Networking, BigQuery, Cloud Run Functions, Cloud Build y Cloud Logging son marginales, lo que indica que los servicios serverless, de integración y monitoreo tienen un impacto económico reducido frente al procesamiento distribuido.

En conjunto, el gráfico evidencia que el principal factor de costo de la arquitectura es el procesamiento computacional, mientras que los servicios de almacenamiento, analítica y automatización presentan una participación secundaria, validando la eficiencia del diseño bajo un esquema de uso controlado de recursos.

3. Diseño del Data Lake

3.1. Diseño y enfoque adoptado

El Data Lake del sistema fue diseñado bajo el patrón Medallion, siguiendo un criterio de separación lógica de capas y trazabilidad de los datos. Si bien el almacenamiento físico de los archivos crudos se realiza en GCS, las capas Bronce, Plata y Oro se implementaron directamente en BigQuery para simplicidad de integración con herramientas de BI como ejemplo, Power BI.

El diseño privilegia un enfoque ETL centralizado en Apache Spark, donde las transformaciones se ejecutan mediante jobs PySpark sobre Dataproc, y los resultados son persistidos en BigQuery utilizando el conector nativo (*write.format("bigquery")*). Esta decisión evita dependencias innecesarias entre servicios y reduce la complejidad operativa del flujo de datos.

Tabla 3. Capas del Data Lake y tecnología utilizada

Capa	Tecnología	Formato	Propósito técnico
Raw	Google Cloud Storage (GCS)	XLS, XLSX, CSV	Almacenamiento del dato original sin transformación
Bronce	BigQuery	Tablas	Ingesta estructurada inicial
Plata	BigQuery	Tablas	Datos limpios y validados
Oro	BigQuery	Tablas dimensionales	Consumo analítico y BI

3.2. Organización lógica de la capa Raw (GCS)

La capa Raw almacena los archivos descargados directamente desde las fuentes oficiales, manteniendo su formato y estructura original. Los datos se organizan por fuente, y cada archivo incluye explícitamente en su nombre el año y mes de referencia, lo que permite una partición lógica basada en nomenclatura sin necesidad de particiones físicas adicionales.

Tabla 4. Reportes regulatorios de la SBS

Código	Tipo de reporte
B-2201	Estados Financieros
B-2315	Créditos por situación
B-2344	Depósitos
B-2370	Patrimonio Efectivo
B-2340	Ratio de Liquidez
B-2402	Patrimonio Requerido RCG

La recolección de datos se realiza de forma automática y periódica mediante una Cloud Function desplegada en Google Cloud Platform (sbs-downloader), la cual ejecuta un script desarrollado en Python. Este componente es responsable de descargar los archivos directamente desde las fuentes oficiales y almacenarlos en la capa Raw del Data Lake.

Tabla 5. Automatización de la descarga de datos

Elemento	Descripción técnica
Servicio	Google Cloud Function
Lenguaje	Python
Frecuencia	Ejecución mensual
Mecanismo	Descarga directa por URL
Control de duplicados	Validación contra archivos existentes
Destino	Google Cloud Storage (capa Raw)

3.3. Implementación de las capas Bronce, Plata y Oro (BigQuery)

3.3.1. Capa Bronce

Cada vez que se incorpora un nuevo archivo a la capa Raw, se activa una Cloud Function event-driven denominada `bronze-dispatcher`. Los datos se convierten a estructuras tabulares estándar, sin aplicar reglas de negocio, preservando el contenido original normalizado.

3.3.2. Capa Plata

La capa Plata contiene los datos limpios y validados, resultado de las transformaciones ejecutadas por Spark. En esta capa se aplican reglas de negocio y normalización de estructuras.

Tabla 6. Configuración del procesamiento distribuido

Componente	Configuración
Plataforma	Google Cloud Dataproc
Tipo de job	PySpark Batch
Ejecutores	2
Núcleos por ejecutor	4
Memoria por ejecutor	4 GB
Driver	4 núcleos / 4 GB
Conector	spark-bigquery

3.3.3. Capa Oro

La capa Oro consolida la información depurada bajo un modelo dimensional tipo estrella, compuesto por tablas de dimensiones y hechos.

4. El modelo dimensional implementado

El modelo dimensional cuenta con una tabla de hechos central (`hecho_riesgo`) y cuatro dimensiones que aportan el contexto temporal, institucional, financiero y analítico necesario para la correcta interpretación de los indicadores de riesgo. Este enfoque optimiza el análisis multidimensional y es ampliamente utilizado en la construcción de data marts orientados a BI.

4.1. Dimensión Fecha (`dim_fecha`)

La dimensión fecha fue diseñada para soportar el análisis temporal de los indicadores de riesgo financiero, permitiendo evaluar su comportamiento histórico, comparativo y tendencial. Esta dimensión proporciona la base cronológica sobre la cual se articulan los análisis mensuales del sistema.

Tabla 7. Descripción de la dimensión fecha

Campo	Descripción
<code>id_fecha</code>	Identificador único de la fecha

anio	Año calendario del registro
mes	Mes calendario del registro
fecha	Fecha representativa del período (primer día del mes)

En el modelo implementado, la dimensión fecha cubre el período comprendido entre enero de 2016 y diciembre de 2025, con una granularidad mensual. Cada registro representa un mes calendario, utilizando como fecha de referencia el primer día del mes, lo que facilita la agregación y comparación de indicadores a lo largo del tiempo.

Posterior a la tabla, esta dimensión permite realizar análisis como evolución interanual, comparaciones por período y evaluación de tendencias de riesgo en horizontes temporales definidos.

4.2. Dimensión Moneda (dim_moneda)

La dimensión moneda permite contextualizar los valores financieros según la divisa utilizada, habilitando análisis multimoneda dentro del modelo analítico. Su inclusión resulta clave para diferenciar información expresada en moneda local y extranjera.

Tabla 8. Descripción de la dimensión moneda

Campo	Descripción
id_moneda	Identificador único de la moneda
abreviacion	Código abreviado de la moneda (ISO-like)
moneda	Nombre descriptivo de la moneda

En el modelo se consideran las monedas Sol Peruano (SOL), Dólar Estadounidense (USD) y Dólar Canadiense (CAD), seleccionadas por su relevancia en el sistema financiero peruano.

Tras la tabla correspondiente, esta dimensión permite segmentar y filtrar indicadores según la moneda de origen, así como soportar análisis comparativos y de conversión cuando sea requerido.

4.3. Dimensión Banco (dim_banco)

La dimensión banco proporciona el contexto institucional necesario para analizar los indicadores de riesgo a nivel de entidad financiera, facilitando comparaciones entre bancos y grupos económicos.

Tabla 9. Descripción de la dimensión banco

Campo	Descripción
id_banco	Identificador único del banco
nombre	Nombre completo de la entidad financiera
abreviacion	Nombre corto o comercial
tipo_entidad	Tipo de institución financiera
grupo_financiero	Grupo económico al que pertenece

El modelo incluye información de las principales entidades del sistema financiero peruano: Scotiabank Perú, Banco de Crédito del Perú (BCP), BBVA Perú, Interbank, BanBif y Mibanco. Para cada banco se almacena su nombre, abreviación, tipo de entidad y grupo financiero, permitiendo análisis tanto individuales como por conglomerado financiero.

Posteriormente a la tabla, esta dimensión habilita comparaciones de desempeño y riesgo entre bancos, así como análisis agregados por tipo de entidad o grupo económico.

4.4. Dimensión Indicador (dim_indicador)

La dimensión indicador constituye el eje analítico del modelo dimensional, ya que define los KPI financieros y de riesgo utilizados para la evaluación del sistema bancario. Esta dimensión no solo describe cada indicador, sino que incorpora su fórmula de cálculo, unidad de medida y umbrales de control, permitiendo interpretar automáticamente los resultados.

Tabla 10. Descripción de la dimensión indicador

Campo	Descripción
id_indicador	Identificador único del indicador
nombre	Nombre del KPI
area_responsable	Área funcional responsable del indicador
formula	Expresión matemática del KPI
descripcion	Interpretación funcional del indicador
unidad	Unidad de medida
lim_amarillo_min	Límite inferior de alerta
lim_amarillo_max	Límite superior de alerta
flag_limite	Indicador de aplicación de umbrales

En esta dimensión se registran indicadores asociados a áreas clave como gestión financiera, gestión de riesgos, créditos, tesorería, gestión comercial y gestión operativa. Los valores almacenados incluyen indicadores de solvencia, liquidez, morosidad, crecimiento crediticio, exposición cambiaria, participación de mercado y pérdidas operativas, todos evaluados con frecuencia mensual.

Tabla 11. Tabla de Ratios Financieros

KPI	Descripción	Fórmula	Unidad	Frecuencia	Nivel de decisión
Loans to Deposits	Relación entre fondos disponibles y créditos concedidos	Colocaciones Brutas / Depósitos	%	Mensual	● $\leq 100\%$ ● 100–120% ● $> 120\%$
Ratio de Capital Global	Solvencia patrimonial del banco	Patrimonio Efectivo / Requerimiento PE / 10%	%	Mensual	● $\geq 10\%$ ● 10–11% ● $< 10\%$

Morosidad	Nivel de créditos atrasados	Créditos atrasados / Créditos Directos	%	Mensual	● $\leq 4\%$ ● 4–8% ● $> 8\%$
Crecimiento de Cartera	Variación interanual de créditos	(Créditos actuales – Créditos año anterior) / Créditos año anterior	%	Mensual	● $> 0\% <$ ● 0 a –5% ● $< -5\%$
Sensibilidad al Tipo de Cambio	Exposición cambiaria del banco	(Activos ME – Pasivos ME) / Patrimonio Neto $\times \% \Delta TC$	%	Mensual	● $< 2\%$ ● 2–5% ● $> 5\%$
Ratio de Liquidez	Cobertura de obligaciones de corto plazo	Activos Líquidos / Obligaciones corto plazo	%	Mensual	● $\geq 100\%$ ● 90–100% ● $< 90\%$
Participación de Depósitos	Cuota de mercado en depósitos	Depósitos banco / Depósitos mercado	%	Mensual	● $> 10\%$ ● 5–10% ● $< 5\%$
Pérdidas Operativas	Impacto de pérdidas operativas	Pérdidas Operativas / (Margen + Ingresos No Financieros)	%	Mensual	● $\leq 2\%$ ● 2–5% ● $> 5\%$

Posterior a la tabla, la información contenida en esta dimensión permite implementar reglas de semaforización (verde, amarillo y rojo), facilitando la toma de decisiones a nivel táctico y estratégico mediante dashboards analíticos.

4.5. Tabla de hechos de eventos de riesgo (hecho_riesgo)

La tabla hecho_riesgo constituye el núcleo transaccional del modelo dimensional, ya que almacena los valores numéricos finales de los indicadores de riesgo financiero calculados a partir de la información procesada en las capas Bronce y Plata del Data Lake.

El llenado de esta tabla se realiza en la capa Oro, donde convergen los datos previamente normalizados, depurados y estandarizados. Tal como se observa en la arquitectura, los archivos provenientes de fuentes operativas internas, información regulatoria (SBS) e indicadores macroeconómicos (BCRP) son inicialmente almacenados en la capa Bronce y posteriormente transformados en la capa Plata mediante procesos PySpark.

Cada registro de la tabla hecho_riesgo representa un evento analítico mensual, identificado por la combinación de claves foráneas hacia las dimensiones dim_banco, dim_fecha, dim_indicador y dim_moneda (Ver Figura 5). Adicionalmente, la tabla almacena los valores numéricos del indicador, desagregados en numerador y

denominador cuando corresponde, lo que permite trazabilidad y recalculabilidad de los KPI.

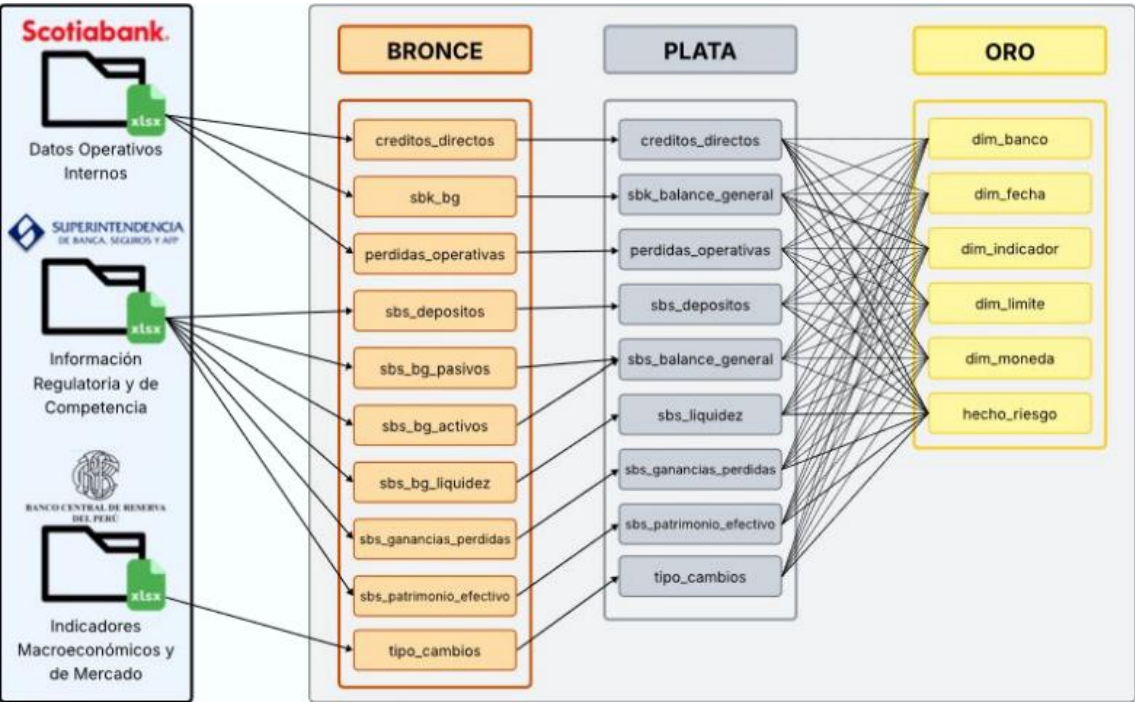


Figura 4. Esquema de datos Medallion aplicada.

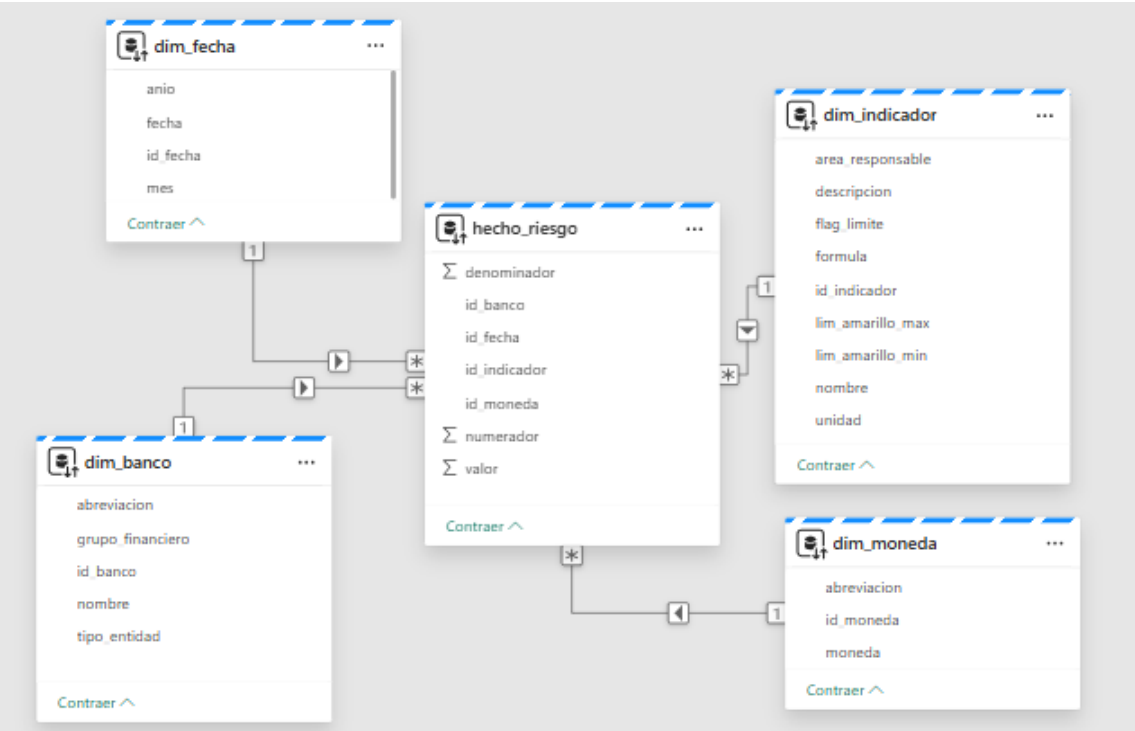


Figura 5. Modelo estrella implementado en la capa Oro

5. Procesos ETL

El flujo ETL se activa a partir de la incorporación de nuevos archivos en la capa Raw, cada archivo corresponde a un tipo de reporte financiero específico y a un periodo determinado, identificado explícitamente en su nombre.

Una vez almacenado el archivo, se genera un evento de almacenamiento que desencadena la ejecución de bronce-dispatcher, responsable de coordinar el procesamiento posterior.

5.1. ETL inicial y carga a la capa Bronce

La función bronce-dispatcher valida la ruta y el tipo de archivo ingresado, determinando el pipeline de procesamiento a ejecutar. A continuación, se invoca un job PySpark en Dataproc que realiza la lectura del archivo fuente y su conversión a una estructura tabular estandarizada.

En esta etapa, el procesamiento se limita a la estructuración básica de los datos, incluyendo la normalización de encabezados, la conversión de tipos primarios y la incorporación de campos de control temporal. El resultado es persistido en BigQuery como tablas pertenecientes a la capa Bronce, manteniendo una versión controlada y auditable de la información original.

5.2. Transformación y depuración en la capa Plata

Finalizada la carga en Bronce, el flujo ETL continúa mediante la ejecución automática de jobs PySpark en Dataproc, los cuales son lanzados por la Cloud Function como parte del mismo flujo orquestado. Cada job está especializado en un dominio de información específico, como créditos, depósitos, patrimonio, tipo de cambio o ratios financieros.

En esta etapa se aplican transformaciones orientadas a la depuración y estandarización avanzada de los datos, incluyendo validaciones de integridad, eliminación de duplicados y armonización de estructuras entre distintos reportes regulatorios. El uso de Apache Spark permite ejecutar estas transformaciones de forma paralela y escalable, garantizando tiempos de procesamiento adecuados.

Los datasets resultantes se almacenan en BigQuery como parte de la capa Plata, constituyendo la base confiable para el análisis financiero.

5.3. Construcción de la capa Oro

La capa Oro se construye a partir de los datos depurados en la capa Plata, mediante jobs PySpark específicos que generan indicadores financieros y estructuras analíticas finales. Previo a la ejecución de estos jobs, se dispone de un proceso de inicialización que crea las dimensiones del modelo analítico, tales como fecha, banco, moneda e indicador, así como la estructura base de la tabla de hechos.

Una vez creada la estructura de la capa Oro, los jobs analíticos consolidan la información por periodo y aplican las fórmulas correspondientes a cada indicador financiero, almacenando los resultados en tablas optimizadas para consultas multidimensionales. Este enfoque permite incorporar nuevos indicadores reutilizando las mismas dimensiones y sin alterar el diseño general del ETL.

6. Los scripts utilizados y consultas SQL

6.1. Servicios GCP

Luego de la creación del proyecto lo primero en implementarse fueron los roles en IAM, para esto se usaron códigos de asignación de permisos para las cuentas de servicio:

```
gcloud projects add-iam-policy-binding grupo6-scotiabank \
  --member="serviceAccount:sbs-scraper-sa@grupo6-
scotiabank.iam.gserviceaccount.com" \
  --role="roles/storage.objectCreator"
gcloud projects add-iam-policy-binding grupo6-scotiabank \
  --member="serviceAccount:75587073872-
compute@developer.gserviceaccount.com" \
  --role="roles/artifactregistry.reader"
gcloud projects add-iam-policy-binding grupo6-scotiabank \
  --member="serviceAccount:service-75587073872@gcp-sa-
eventarc.iam.gserviceaccount.com" \
  --role="roles/storage.objectViewer"
```

De forma adicional se hizo lo análogo para la asignación de permisos para los usuarios 1 y 2 de perfiles de desarrollo con permisos específicos acorde a su rol y funciones.

```
gcloud projects add-iam-policy-binding grupo6-scotiabank --member="user:PONER@USUARIO1" --role="roles/cloudfunctions.developer"
gcloud projects add-iam-policy-binding grupo6-scotiabank --member="user:PONER@USUARIO1" --role="roles/storage.admin" && \
gcloud projects add-iam-policy-binding grupo6-scotiabank --member="user:PONER@USUARIO1" --role="roles/cloudscheduler.admin" && \
gcloud projects add-iam-policy-binding grupo6-scotiabank --member="user:PONER@USUARIO1" --role="roles/iam.serviceAccountUser" && \
gcloud projects add-iam-policy-binding grupo6-scotiabank --member="user:PONER@USUARIO1" --role="roles/run.admin" && \
gcloud projects add-iam-policy-binding grupo6-scotiabank --member="user:PONER@USUARIO1" --role="roles/resourceManager.projectIa

gcloud projects add-iam-policy-binding grupo6-scotiabank --member=user:PONER@USUARIO2 --role="roles/bigquery.dataOwner" && \
gcloud projects add-iam-policy-binding grupo6-scotiabank --member=user:PONER@USUARIO2 --role="roles/cloudfunctions.developer" && \
gcloud projects add-iam-policy-binding grupo6-scotiabank --member=user:PONER@USUARIO2 --role="roles/storage.admin" && \
gcloud projects add-iam-policy-binding grupo6-scotiabank --member=user:PONER@USUARIO2 --role="roles/cloudscheduler.admin" && \
gcloud projects add-iam-policy-binding grupo6-scotiabank --member=user:PONER@USUARIO2 --role="roles/run.admin"
```

Figura 6. Códigos para brindar permisos necesarios a los usuarios

Después de eso se integran todos los demás servicios que se usaron en el proyecto


```
# Habilitar todas las APIs de una vez
gcloud services enable \
    storage.googleapis.com \
    dataproc.googleapis.com \
    bigquery.googleapis.com \
    cloudfunctions.googleapis.com \
    cloudscheduler.googleapis.com \
    monitoring.googleapis.com \
    logging.googleapis.com

# Esperar 2-3 minutos para que se activen
# Verificar que estén habilitadas
gcloud services list --enabled
```

Figura 7. Implementación de los servicios necesarios para el proyecto

Para facilitar el proceso de creación de los servicios se estructuraron 3 fases del proceso los cuales tienen un script de creación e implementación de todos los servicios que son necesarios para esa fase. Un ejemplo es la parte de extracción y carga de los archivos desde la SBS, en esta parte se verifica la existencia de un bucket y se crea una service account para cloud functions para después crear la función correspondiente en el servicio

```
# 3. Crear bucket si no existe
echo "🔍 Verificando bucket..."
if ! gsutil ls gs://$BUCKET_NAME > /dev/null 2>&1; then
    echo "Creando bucket $BUCKET_NAME..."
    gsutil mb -p $PROJECT_ID -l $REGION gs://$BUCKET_NAME
    echo "✓ Bucket creado"
else
    echo "✓ Bucket ya existe: $BUCKET_NAME"
fi

# Nota: Las carpetas en GCS se crean automáticamente cuando subes el primer archivo
# No es necesario crear carpetas manualmente

# 4. Crear Service Account para la Cloud Function
echo "🔑 Configurando Service Account para la función..."
if ! gcloud iam service-accounts describe $SA_EMAIL > /dev/null 2>&1; then
    echo "Creando Service Account: $SA_NAME..."
    gcloud iam service-accounts create $SA_NAME \
        --display-name="SBS Downloader Function Service Account" \
        --description="Service Account para ejecutar la Cloud Function de descarga SBS"
    echo "✓ Service Account creado"
else
    echo "✓ Service Account ya existe: $SA_NAME"
fi
```

Figura 8. Implementación de Bucket y Cloud Function

```
# 6. Desplegar Cloud Function (2nd gen) con Service Account
echo "🚀 Desplegando Cloud Function..."
gcloud functions deploy $FUNCTION_NAME \
  --gen2 \
  --region=$REGION \
  --runtime=python311 \
  --source=. \
  --entry-point=sbs_downloader_http \
  --trigger-http \
  --no-allow-unauthenticated \
  --service-account=$SA_EMAIL \
  --timeout=540s \
  --memory=512MB \
  --max-instances=1 \
  --set-env-vars=BUCKET_NAME=$BUCKET_NAME

echo "✓ Cloud Function desplegada"
```

Figura 9. Implementación de función en Cloud Function

De igual manera se tienen códigos para la implementación de todos los servicios, archivos y rutas necesarias para el funcionamiento del proceso

6.2. Medallion

Para la transformación de los datos en cada una de las capas se usó código en Python en la generación de las 3 capas Medallion.

Para garantizar la replicabilidad de la solución, se desarrolló un script en Shell (.sh) que automatiza todo el proceso de configuración en Google Cloud Platform. Este script se encarga de habilitar las 15 APIs necesarias (Dataproc, BigQuery, Pub/Sub, Eventarc, Storage, entre otras), crear en el bucket de almacenamiento, la estructura de carpetas, subir los scripts PySpark al bucket.

Asimismo, se configura todos los permisos de IAM requeridos para que los diferentes servicios de GCP puedan comunicarse correctamente. Específicamente, asigna los roles necesarios a las service accounts de Eventarc y Cloud Storage, limpia notificaciones previas para evitar conflictos de cuota.

Además se ejecuta un job para crear la estructura de la capa oro y, finalmente, se despliega la Cloud Function de segunda generación llamada bronce-dispatcher que actúa como disparador del pipeline, como se explicó en la sección anterior.

```

1  #!/bin/bash
2  set -e
3
4  # -----
5  # CONFIGURACIÓN GENERAL
6  # -----
7  PROJECT_ID="grupo6-scotiabank"
8  REGION="southamerica-west1"
9  BUCKET_NAME="grupo6-scotiabank_bucket" # Modificar si es necesario
10
11 echo "Proyecto: $PROJECT_ID"
12 echo "Región: $REGION"
13 echo "Bucket: $BUCKET_NAME"
14
15 gcloud config set project $PROJECT_ID
16
17 # -----
18 # OBTENER PROJECT NUMBER
19 # -----
20 PROJECT_NUMBER=$(gcloud projects describe $PROJECT_ID \
21 --format="value(projectNumber)")
22
23 EVENTARC_SA="service-${PROJECT_NUMBER}@gcp-sa-eventarc.iam.gserviceaccount.com"
24
25 echo "Eventarc Service Account: $EVENTARC_SA"
26
27 # -----
28 # 1. HABILITAR APIS
29 # -----
30 echo "Habilitando APIs necesarias..."
31
32 gcloud services enable \
33 | cloudfunctions.googleapis.com \
34 | cloudbuild.googleapis.com \
35 | artifactregistry.googleapis.com \
36 | run.googleapis.com \
37 | pubsub.googleapis.com \
38 | eventarc.googleapis.com \
39 | dataproc.googleapis.com \
40 | dataproc-control.googleapis.com \
41 | storage.googleapis.com \
42 | bigquery.googleapis.com \
43 | bigquerystorage.googleapis.com \
44 | logging.googleapis.com \
45 | monitoring.googleapis.com
46
47 # -----
48 # 2. CREAR BUCKET (SI NO EXISTE)
49 # -----
50 echo "Creando bucket $BUCKET_NAME..."
51
52 gsutil mb -p $PROJECT_ID -l $REGION gs://$BUCKET_NAME || echo "Bucket ya existe"
53
54 # Crear carpetas lógicas
55 gsutil mkdir gs://$BUCKET_NAME/resources || true
56 gsutil mkdir gs://$BUCKET_NAME/raw || true
57
58 # -----
59 # 3. SUBIR SCRIPTS SPARK
60 # -----
61 echo "Subiendo jobs..."
62
63 gsutil cp jobs/*.py gs://$BUCKET_NAME/resources/
64
65 -----
66 4. EJECUTAR JOB SPARK
67 -----
68 echo "Ejecutando Dataproc Serverless job..."
69
70 gcloud dataproc batches submit pyspark \
71 | gs://$BUCKET_NAME/resources/crear_capa_oro.py \
72 | --project=$PROJECT_ID \
73 | --region=$REGION \
74 | --batch=create-oro-tables2 \
75 | --version=2.1 \
76 | --jars=gs://spark-lib/bigquery/spark-bigquery-latest_2.12.jar \
77 | --properties=spark.executor.instances=2,spark.executor.cores=4,spark.executor.memory=4g,spark.driver.cores=4,spark.driver.memory=4g

```

Figura 10. Script en Shell para desplegar bronce- dispatcher

Para la etapa de Bronce se generaron códigos que procesaban los archivos desde el formato de la SBS a un formato tabla a través de un job en Dataproc.

Code

Blame

168 lines (130 loc) · 4.78 KB

```
1  from google.cloud import dataproc_v1
2  from google.cloud.dataproc_v1.types import Batch, PySparkBatch, RuntimeConfig
3  import logging
4  from datetime import datetime
5  import pandas as pd
6  from utils.gcs import leer_excel_desde_gcs
7  from utils.bigquery import cargar_dataframe_bigquery
8  from config.paths import DATASET_BRONCE, TABLE_credito
9  import re
10
11
12  PROJECT_ID = "grupo6-scotiabank"
13  REGION = "southamerica-west1"
14  BUCKET_SCRIPT = "gs://grupo6_scotiabank_bucket/resources/jb_creditos.py"
15
16  BATCH_NAME = "jb_creditos"
17  timestamp = datetime.now().strftime("%Y%m%d-%H%M%S")
18  BATCH_NAME = f"{BATCH_NAME}-{timestamp}"
19
20
21
22  def run_pipeline_creditos(bucket_name, file_name):
23      logging.info(f"[PIPELINE] Procesando creditos -> {file_name}")
24
25      df_raw = leer_excel_desde_gcs(bucket_name, file_name, "36")
26      if df_raw is None:
27          logging.error("No se pudo leer el Excel.")
28          return
29
30      df_transformado = _transformar_creditos(df_raw, file_name)
31      if df_transformado is None:
32          logging.error("No se pudo transformar los datos.")
```

Figura 11. Job de transformación para la capa de bronce para los créditos

De la misma manera se elaboraron códigos de procesamiento para las capas de plata y oro, que filtran, consolidan y calculan los indicadores por cada unidad temporal.

```

1  from pyspark.sql import SparkSession
2  from pyspark.sql.functions import col, concat_ws, lpad, to_date, lit, row_number
3  from pyspark.sql.window import Window
4  from google.cloud import bigquery
5  from pyspark.sql.types import StructType, StructField, IntegerType, DoubleType, StringType
6
7  # -----
8  # CONFIGURACIÓN
9  # -----
10 PROJECT_ID = "grupo6-scotiabank"
11 DATASET = "oro"
12 DATASET_REGION = "southamerica-west1"
13
14
15
16 # -----
17 # CREAM DATASET SI NO EXISTE
18 # -----
19 ✓ def ensure_dataset_exists():
20     client = bigquery.Client(project=PROJECT_ID)
21
22     dataset_full_id = f"{PROJECT_ID}.{DATASET}"
23
24     try:
25         client.get_dataset(dataset_full_id)
26         print(f"Dataset {dataset_full_id} ya existe.")
27     except Exception:
28         dataset = bigquery.Dataset(dataset_full_id)
29         dataset.location = DATASET_REGION
30         client.create_dataset(dataset, exists_ok=True)
31         print(f"Dataset {dataset_full_id} creado correctamente.")
32

```

Figura 12. Creación del dataset de capa oro en bigquery.

```

1  from pyspark.sql import SparkSession
2  from pyspark.sql.functions import (
3      col, when, to_date, concat_ws, lpad, lit,
4      lower, trim, create_map, coalesce
5  )
6  from itertools import chain
7  from google.cloud import bigquery
8  from pyspark.sql import functions as F
9  from pyspark.sql.types import IntegerType, DoubleType, StringType
10 import re
11
12
13 # -----
14 # Inicializar Spark
15 # -----
16 spark = SparkSession.builder.appName("PlataDepositos").enableHiveSupport().getOrCreate()
17
18
19 # -----
20 # Parámetros BigQuery
21 # -----
22 project_id = "grupo6-scotiabank"
23 dataset_plata = "plata"
24 dataset_oro = "oro"
25 table_plata = "sbs_deposito"
26 table_oro = "hecho_riesgo"
27 minutos = 10
28
29
30 # -----
31 # Oro - IDs fijos
32 # -----
33 ID_MONEDA = 1
34 ID_INDICADOR = 7
35 ID_LIMITE = 7

```

Figura 13. Procesamiento de datos en las capas plata y oro.

7. Visualizaciones Finales Implementadas

Las visualizaciones finales del sistema fueron implementadas utilizando Microsoft Power BI Desktop, conectado directamente al motor analítico Google BigQuery, donde se encuentra desplegado el modelo dimensional en la capa Oro del Data Lake. La decisión de emplear Power BI responde a su capacidad para integrarse de forma nativa con BigQuery, soportar modelos semánticos complejos y habilitar análisis interactivos orientados a la toma de decisiones financieras.

La conexión se realiza mediante una Cuenta de Servicio de Google Cloud, configurada en Power BI como mecanismo de autenticación, lo que permite:

- Acceso seguro sin uso de credenciales personales.
- Control granular de permisos mediante IAM.
- Reproducibilidad del dashboard en distintos entornos.

Las visualizaciones implementadas se agrupan en:

- **Indicadores clave (KPIs)**
 - Ratios financieras consolidados.
 - Indicadores de liquidez, solvencia y patrimonio.
 - Valores agregados por periodo y entidad.
 - Filtros por banco, fecha, moneda e indicador.



Figura 14. Tablero principal de apetito por cada tipo de riesgo

- **Análisis de riesgo**
 - Monitoreo temporal de indicadores financieros mediante semáforos de riesgo.
 - Visualización de la evolución mensual de créditos, depósitos y ratios financieros.
 - Identificación temprana de escenarios de deterioro o mejora financiera.



Tablero de Apetito por Riesgo

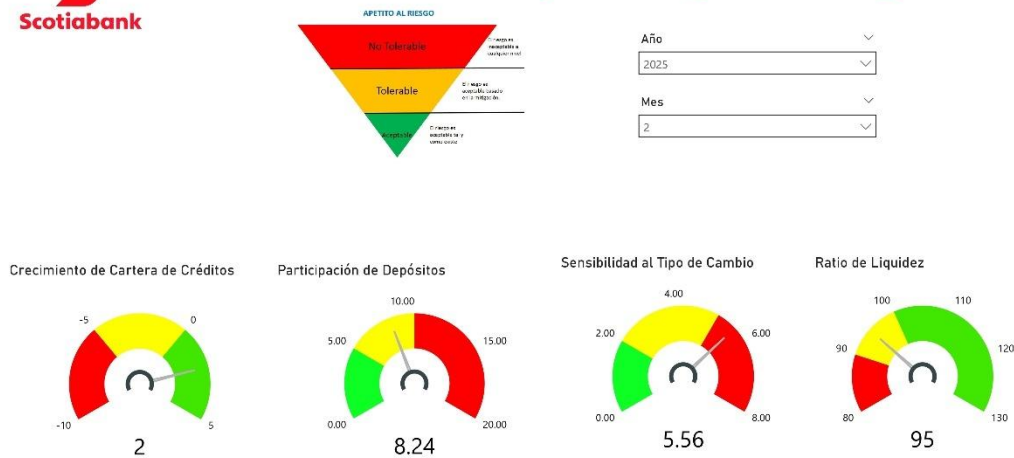


Figura 15. Tablero 1 de KPIs de riesgo por temporalidad



Tablero de Apetito por Riesgo



Figura 16. Tablero 2 de KPIs de riesgo por temporalidad

- **Análisis comparativo**
 - Comparación entre entidades financieras.
 - Evaluación de participación relativa por indicador.
 - Análisis cruzado entre variables financieras y temporales.



Figura 17. Evolutivo histórico de cada indicador por banco

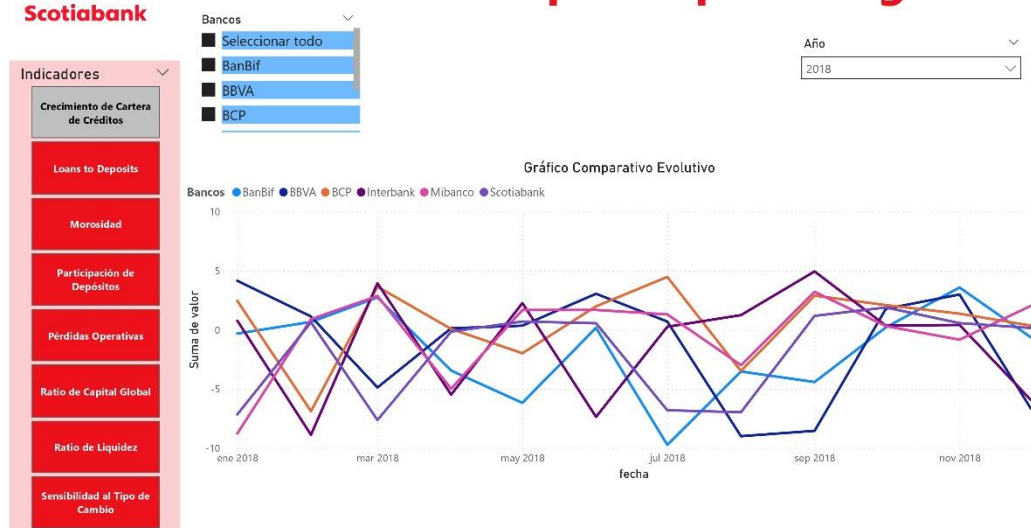


Figura 18. Comparativo de la evolución anual de un indicador para varios bancos

8. El uso del repositorio GitHub.

La distribución inicial de las ramas fue por roles asignados de la siguiente manera:

Tabla 12. Tabla de roles y ramas por estudiante

Estudiante	Rol	Rama	Proceso
Álvarez Casas Julio Cesar	Líder de Equipo	feature/grupo06-eda- inicial	Mostrar datos (Tablero en Power BI)
Campos Herrera Dennis Leopoldo	Analista de datos	feature/grupo06- limpieza-datos	Obtención de datos desde la página de la SBS al Bucket
Espinal Aguilar, Albert Moisés	Analista de datos	feature/grupo06- analisis-datos	Procesamiento de archivos descargados a datos procesados (Capa Oro)

Al inicio del sprint realizábamos el merge desde la rama principal hacia cada una de nuestras ramas para empezar con la misma estructura, posteriormente desarrollábamos nuestras labores en cada una de nuestras ramas documentando nuestros procesos, finalmente realizábamos el merge hacia la rama del líder quien era el encargado de hacer los merges a la rama principal “feature/grupo06-init”.

Branch	Updated	Check status	Behind	Ahead	Pull request
feature/grupo06-eda-inicial	3 hours ago		50	15	#114
feature/grupo06-PC4	last week		69	7	#108
feature/grupo06-limpieza-datos	2 weeks ago		77	0	#113
feature/grupo06-analisis-datos	2 weeks ago		77	0	#112
	2 weeks ago		286	28	#103
feature/grupo06-init	last month		243	0	#46

Figura 19. Ramas en Github relacionadas al trabajo del grupo

BIBLIOGRAFÍA

Banco Central de Reserva del Perú. (s. f.). *Ratio de liquidez promedio (%) – Scotiabank Perú S.A.A.* [Serie estadística mensual]. BCRPData.
<https://estadisticas.bcrp.gob.pe/estadisticas/series/mensuales/resultados/PN07187NM/html>

Cloudera. (2015). *Hortonworks Data Platform – Getting Started Guide (HDP 2.1.2)*. Recuperado de https://docs.cloudera.com/HDPDocuments/HDP2/HDP-2.1.2-Win/bk_getting-started-guide/content/ch_about-hortonworks-data-platform.html

Dawar, R. (2022, Agosto). *Data Mart vs. Data Warehouse: Should you use either or both?* <https://atlan.com/data-mart-vs-data-warehouse/>

Montalbán. (2020). *Guía de instalación: Hortonworks Data Platform*. Universitat Oberta de Catalunya. Recuperado de:
<https://openaccess.uoc.edu/server/api/core/bitstreams/023dcf3b-b8a-9aae-8a01ab0e2348/content>

Scotiabank. (2025). *Scotiabank Perú*. Recuperado el 22 de septiembre de 2025, de <https://www.scotiabank.com.pe/>

Superintendencia del Mercado de Valores. (2025). *Scotiabank Perú S.A.A.: Estados financieros separados al 31 de diciembre de 2024 y de 2023 (con el informe de los auditores independientes)*.
<https://www.smv.gob.pe/ConsultasP8/temp/EEFF%20Separado%20SBP%202024.pdf>

Superintendencia de Banca, Seguros y AFP. (s. f.). *Formación estadística de banca múltiple*.
https://www.sbs.gob.pe/app/stats_net/stats/EstadisticaBoletinEstadistico.aspx?p=1

ANEXOS

Repositorio del informe:

https://github.com/SI807-FIIS-UNI/SI807_Cloud_BI_2025